Which Small Model Guards the Gate? A Pilot Benchmark, Generational Bridge Experiment, and Pre-Registered Four-Arm RCT Design for Production Front-Door Routing

Warren Johnson*
Plexor Labs
Sammamish, WA, USA

Charles Lee
Project Autobots
Seattle, WA, USA

Abstract

Production LLM inference systems increasingly deploy a *front-door router*—a small language model (SLM) that classifies incoming prompts by task type and complexity, then routes each request to an appropriately capable (and priced) downstream model. The router is itself a model-selection problem: cheaper SLMs offer low latency but may fail at nuanced task-family boundaries; stronger SLMs improve classification quality but violate latency budgets or introduce external API dependencies. We report two completed studies and one pre-registered design for the PLEXOR front-door stack. **Study 1 (Pilot Benchmark):** A locked, hardware-controlled pairwise comparison of Phi-3.5-mini (3.8B) and Qwen2.5-1.5B on a fixed 60-case corpus with a pre-registered JSON-safe decoding contract. On RunPod Secure A40, Phi-3.5-mini achieves significantly higher exact-label accuracy (0.517 vs. 0.200; exact McNemar $p = 0.0013$, $\Delta = +0.317$, 95% CI $[0.15, 0.48]$), while Qwen2.5-1.5B is substantially faster (median latency $1{,}017$ ms vs. $12{,}227$ ms; Wilcoxon $p = 1.63 \times 10^{-11}$). Family-level analysis reveals a *split policy*: Phi dominates code-complex and chain-of-thought families; Qwen dominates hybrid-agentic prompts. **Study 2 (Generational Bridge):** A same-prompt evaluation of Qwen-2.5-3B-Instruct (3B, 4-bit quantized, vLLM 0.17.1) on a reconstructed version of the same 60-case corpus (SHA `dac3aac5`), executed on an Azure T4 GPU. Qwen-2.5-3B achieves 0.767 overall accuracy ($46/60$), a $+0.567$ absolute improvement over its 1.5B predecessor and a $+0.250$ improvement over Phi-3.5-mini, with perfect parse rate and $1{,}564$ ms mean latency. **Study 3 (Four-Arm RCT Design):** A pre-registered randomized controlled trial comparing three candidate SLM backends—Phi-4-mini-instruct (Azure AI Foundry serverless), Qwen-2.5-3B-Instruct (self-hosted vLLM), and DeepSeek-V3 (commercial API)—against a no-routing control, with $N \geq 250$ per arm, Bonferroni-corrected pairwise tests ($\alpha/6 = 0.0083$), O'Brien-Fleming interim analysis at $N = 125$, and a pre-registered decision matrix with Pareto frontier analysis. Design complete; execution pending. Study 3 presents the pre-registered protocol; a companion paper will report execution results. Together, these studies provide an end-to-end empirical path from offline model comparison through generational validation to production-grade experimentation. All run IDs, corpus hashes, and reproducibility manifests are released.

Keywords: model routing, small language models, front-door classification, randomized controlled trial, MLflow, latency-quality tradeoff, Pareto frontier, reproducibility

---

Series position. This is Article 8 in the TAAC Research Series. Articles 1–3 established task-dependent compression thresholds and the perplexity paradox mechanism (Johnson, 2026c;f;b). Articles 4–5 documented provider-dependent output token explosion under compression (Johnson, 2026e;d). Article 6 validated benchmark-derived thresholds in a production RCT ($N = 358$, six arms, CONSORT reporting) (Johnson and Lee, 2026). Article 7 designed tokenizer-aware prompt languages achieving up to 64.7% token reduction (Johnson, 2026a). This article shifts from *what to*

*Corresponding author. E-mail: `warrenjo@plexor.dev`

compress to *which model should decide*—extending the "route" branch of the Article 1 dichotomy into model selection.

1 Introduction

The cost structure of large language model (LLM) inference creates a natural incentive for *front-door routing*: before an expensive frontier model processes a request, a cheaper model classifies the incoming prompt by task type and complexity, then routes the request to an appropriately capable—and appropriately priced—downstream model (Jiang et al., 2024b; Shnitzer et al., 2023; Chen et al., 2023b). This architecture has become standard in production LLM deployments, where a single percentage point of routing accuracy can translate to thousands of dollars per day in inference cost savings (Lu et al., 2023). The underlying economic logic is straightforward: if a $0.001-per-query classifier can correctly identify that 40% of incoming traffic requires only a $0.01-per-query model rather than a $0.10-per-query frontier model, the cost savings compound rapidly at scale. At 100,000 queries per day, even a modest routing accuracy improvement of 5 percentage points yields annualized savings exceeding $150,000, dwarfing the classifier's own operational cost.

The front door is itself a model-selection problem. Cheaper small language models (SLMs) in the 1–4B parameter range offer sub-second latency but can fail at nuanced task-taxonomy boundaries; stronger SLMs or even full-scale models improve classification quality but may violate latency budgets, introduce external API dependencies, or negate cost savings by consuming the very inference budget they are meant to protect (Chen et al., 2023b). This tradeoff is amplified when teams require both *scientific validity*—paired significance tests, dataset immutability, reproducibility—and *operational validity*—health checks, logging schema, alertability, arm isolation. The tension between these two modes of validity is rarely acknowledged in the routing literature, where most work evaluates routing quality in isolation from the operational infrastructure that must sustain it in production (Madaan et al., 2023).

The challenge is further compounded by the rapid pace of SLM development. The transformer architecture (Vaswani et al., 2017) has been adapted into increasingly efficient small-scale variants, from the Phi family's textbook-quality training data approach (Gunasekar et al., 2023; Li et al., 2023; Javaheripi et al., 2023; Abdin et al., 2024a;b) to the Qwen family's multilingual scaling strategy (Yang et al., 2024a;b) and the open-weight ecosystem of Llama (Touvron et al., 2023; Meta AI, 2024), Gemma (Gemma Team, 2024; Gemma Team et al., 2024), Mistral (Jiang et al., 2023; 2024a), TinyLlama (Zhang et al., 2024a), StableLM (Zhu et al., 2024), and the TinyStories line of research demonstrating surprisingly coherent generation from sub-billion-parameter models (Eldan and Li, 2023). Each new release reshuffles the Pareto frontier of quality versus cost, rendering previously optimal routing configurations suboptimal within months. This "model churn" problem means that any static evaluation of routing models is inherently time-limited—a finding that motivates our multi-study design, which progresses from a locked offline benchmark through a generational bridge experiment to a pre-registered RCT that can accommodate future model candidates.

Article 1 in this series (Johnson, 2026c) demonstrated a fundamental dichotomy: code-generation tasks benefit from compression-first strategies (Cohen's $d = 1.84$ over 2,650 trials), while chain-of-thought reasoning benefits from routing to appropriately-capable models. That finding established *routing* as a first-class cost-reduction mechanism parallel to compression. But it left open the question of *which model performs the routing itself*—the subject of the present article. The distinction is important: while prior work on learned routing (Jiang et al., 2024b; Ong et al., 2025) trains a dedicated classifier on preference data, and cascade architectures (Madaan et al., 2023; Yue et al., 2024) invoke multiple models per query, our front-door approach requires a single SLM to make an interpretable, discrete classification decision using only the raw prompt text. This constraint—single model, single pass, discrete output—places unique demands on SLM capability that differ from the generative and reasoning tasks on which SLMs are typically benchmarked (Hendrycks et al., 2021; Clark et al., 2018; Zellers et al., 2019; BigCode Team, 2023; Austin et al., 2021). The question of which SLM should serve as the front-door classifier also intersects with emerging work on test-time compute scaling (Snell et al., 2024) and speculative decoding (Leviathan et al., 2023; Chen et al., 2023a), suggesting that the traditional tradeoff between model size and inference speed may be more nuanced than a simple parameter-count comparison implies.

We contribute three complementary studies spanning the full path from offline evaluation through production deployment design: (1) a **locked offline benchmark** (Study 1, §2–§3) comparing two SLMs under hardware-controlled conditions with paired statistical tests on a 60-case corpus; (2) a **generational bridge experiment** (Study 2, §4–§5) evaluating whether a single generation jump in the Qwen family (1.5B $\rightarrow$ 3B) resolves the accuracy deficit observed in Study 1; and (3) a **pre-registered four-arm RCT design** (Study 3, §6) with Bonferroni correction, O'Brien-

Fleming interim analysis, and a Pareto frontier decision matrix. Beyond the empirical findings, this paper makes three methodological contributions: a locked-contract evaluation protocol that eliminates confounding variation (§2), drawing on best practices from clinical trial design (Schulz et al., 2010) and trustworthy online experimentation (Kohavi et al., 2020; 2009); a cross-generation transferability analysis for assessing whether findings from one model generation transfer to the next (§4); and a pre-registered multi-arm RCT adapted from clinical trial methodology, including Bonferroni correction for six hypotheses, O'Brien-Fleming interim boundaries (O'Brien and Fleming, 1979), and sequential testing principles (Wald, 1945).

1.1 The Model Selection Problem for Front-Door Routing

The model routing problem has emerged as a central challenge in production LLM deployment, and we identify four dominant architectural paradigms. *Learned routers* train on preference data to predict which model will produce a better response for a given prompt. Jiang et al. (2024b) achieve $2\times$ cost reduction on MT-Bench while maintaining 95% of GPT-4 quality with a BERT-scale classifier (Devlin et al., 2019); Ong et al. (2025) extend this to an open-source framework; and Hu et al. (2024) provide a comprehensive benchmark for evaluating such multi-LLM routing systems. *Benchmark-guided routing* (Shnitzer et al., 2023) selects the model with the highest predicted performance on a prompt's task type using benchmark scores as a prior, building on holistic evaluation frameworks (Liang et al., 2023; Srivastava et al., 2023). *Cascade architectures* (Chen et al., 2023b; Madaan et al., 2023; Yue et al., 2024; Lu et al., 2023) invoke cheaper models first and escalate to expensive models only when confidence is low, differing from our single-pass front-door approach. *Meta-modeling approaches* (Šakota et al., 2024; Zhang et al., 2024b) predict cost-effectiveness of each candidate for a given input; these are conceptually closest to our work but operate at the model-selection level rather than the task-classification level. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023a), Branch-Train-Merge (Li et al., 2022), and Mixture-of-Agents (Wang et al., 2024b) address inference efficiency through parallel execution rather than routing.

Our work addresses a specific variant that we term *front-door classification*: a single SLM classifies each incoming prompt into a discrete task taxonomy, and the taxonomy label deterministically selects the downstream model and compression policy. This differs from learned routing in three ways. First, the classification label is human-readable, auditable, and overridable—learned routers produce opaque preference scores. Second, the taxonomy label can trigger multiple downstream decisions (model tier, compression ratio, output budget, quality gate) through a lookup table, whereas learned routers optimize for a single downstream metric. Third, the taxonomy is version-controlled and can be updated without retraining. The tradeoff is that front-door classification requires a well-defined taxonomy with clear boundaries; our 6-family taxonomy (`code/simple`, `code/complex`, `CoT/simple`, `CoT/complex`, `hybrid/agentic`, `hybrid/generative`) is derived from the Article 1 dichotomy extended with complexity and modality dimensions.

SLMs as classifiers. The use of SLMs in the 1–4B parameter range as task classifiers is motivated by several factors: classification requires understanding prompt *intent* rather than generating a complete response, potentially requiring less parametric capacity (Kaplan et al., 2020); the output budget is minimal (a single JSON label), eliminating the output token dynamics that dominate inference cost for generative tasks (Willard and Louf, 2023; Zheng et al., 2024b); and SLMs can be self-hosted on commodity GPUs, avoiding API dependencies and data-residency concerns. However, SLMs face known limitations. The Phi family (Abdin et al., 2024a;b) achieves strong performance on structured tasks but may struggle with ambiguous prompts. The Qwen family (Yang et al., 2024a;b) exhibits strong multilingual capabilities but has received less evaluation on classification-specific tasks. DeepSeek's MoE architecture (Liu et al., 2024; Dai et al., 2024) activates only 37B of its 671B parameters per token, but the internal routing decision may interact with the classification task unpredictably (Shazeer et al., 2017; Fedus et al., 2022; Lepikhin et al., 2021). Recent surveys of small language models (Lu et al., 2024; Wang et al., 2024a) provide comprehensive coverage but do not specifically address classification-for-routing. The Orca (Mukherjee et al., 2023; Mitra et al., 2023) and WizardLM (Xu et al., 2023) lines of work demonstrate that smaller models can acquire sophisticated reasoning through progressive learning from stronger models, and parameter-efficient adaptation methods such as prefix tuning (Li and Liang, 2021) and prompt tuning (Lester et al., 2021) offer low-cost specialization paths. Production deployment of SLM classifiers requires 4-bit quantization to fit within GPU memory budgets on commodity hardware (T4, A10); recent work on QLoRA (Dettmers et al., 2023), GPTQ (Frantar et al., 2023), AWQ (Lin et al., 2024), and SmoothQuant (Xiao et al., 2023) has demonstrated that 4-bit quantization preserves most quality for generative tasks, but the impact on classification accuracy specifically is less well-characterized.

1.2 Research Questions and Hypotheses

The overarching research question is: *Which small language model backend maximizes front-door classification accuracy while satisfying latency and cost constraints for production routing?* We decompose this into hypotheses organized by study.

Study 1 hypotheses (validated).

Hypothesis 1 (H1: Quality Advantage)**.** Under the locked `jsonsafe_cls_tokens_128` contract, Phi-3.5-mini achieves significantly higher exact-label accuracy than Qwen2.5-1.5B on the 60-case corpus.

Hypothesis 2 (H2: Latency Advantage)**.** Qwen2.5-1.5B achieves significantly lower per-case inference latency than Phi-3.5-mini under identical hardware conditions.

Hypothesis 3 (H3: Task-Family Interaction)**.** The quality advantage is not uniform across task families: different SLMs dominate different families, producing a "split policy" rather than a single winner.

Study 2 hypotheses (validated).

Hypothesis 4 (H4: Generational Jump)**.** Qwen-2.5-3B-Instruct achieves higher classification accuracy than both Qwen2.5-1.5B and Phi-3.5-mini from Study 1, resolving the accuracy deficit while preserving sub-2-second latency.

Hypothesis 5 (H5: Family Coverage)**.** The generational jump improves accuracy on families where Qwen2.5-1.5B scored 0.0 (CoT/simple, CoT/complex, code/simple), narrowing the family-level performance gap.

Study 3 hypotheses (pre-registered, pending execution).

Hypothesis 6 (H6: Routing Accuracy)**.** Routing accuracy differs across arms, with expected ordering $D > C \geq B > A$. Success threshold: $\geq 85\%$ for the winning arm.

Hypothesis 7 (H7: Total Cost)**.** Average USD per request (front-door + downstream) differs, with B or $C < D < A$. Threshold: $\geq 25\%$ reduction vs. Arm A.

Hypothesis 8 (H8: Classification F1)**.** F1 macro and hybrid/agentic-specific F1 differ, with $D > C \geq B$. Threshold: F1 macro ≥ 0.82.

Hypothesis 9 (H9: Latency SLA)**.** P95 front-door latency differs, with $C \leq B < D$ (network penalty). Hard gate: ≤ 200 ms for all arms.

Hypothesis 10 (H10: Compression Quality)**.** Average compression ratio at semantic similarity ≥ 0.88 differs by model backend. Threshold: $\geq 30\%$ token reduction.

2 Methods: Study 1 (Pilot Benchmark)

The pilot study uses a locked corpus and configuration to establish baseline quality-latency profiles for two candidate SLMs under identical conditions. This section describes the experimental contract—the complete specification that must be held invariant for results to be valid—the models under evaluation, the classification system prompt, and the statistical analysis plan.

Formal problem statement. We define the front-door classification task as follows. Let $\mathcal{P}$ denote the space of natural-language prompts and let the label set be

$$\mathcal{L} = \{\texttt{cd/smp},\ \texttt{cd/cplx},\ \texttt{CoT/smp},\ \texttt{CoT/cplx},\ \texttt{hyb/agnt},\ \texttt{hyb/gen}\}.$$

A front-door classifier is a function

$$f : \mathcal{P} \longrightarrow \mathcal{L} \times [0, 1], \tag{1}$$

mapping each prompt p to a predicted label $\hat{\ell} = f(p)_{\text{label}} \in \mathcal{L}$ and a self-reported confidence $c = f(p)_{\text{conf}} \in [0, 1]$. The downstream routing table $R : \mathcal{L} \to \mathcal{M} \times \Theta$ maps each label to a (model, compression-policy) pair, making the classification decision the single point of control for the entire inference pipeline.

Experimental contract. The contract identifier is `jsonsafe_cls_tokens_128`. The corpus is `progressive_test_cases_v2_60` (60 prompts, 6 families $\times$ 10 cases each; SHA-256: `3be755e8...`). Decoding: `max_new_tokens=128` (classifier), 24

(scorer), $T = 0.0$ (greedy). Quantization is 4-bit NF4 via bitsandbytes for both models. Fairness invariants require the same host class, commit hash, parser path, and no mid-pair drift. The two models represent opposite ends of the SLM design space: Phi-3.5-mini-instruct (3.8B, dense, Microsoft) optimizes for instruction-following quality; Qwen2.5-1.5B-Instruct (1.5B, dense, Alibaba) optimizes for minimal footprint.

Classification system prompt. Both models receive an identical, frozen classification system prompt:

```
You are a task classifier for an AI routing system.
Classify the prompt into exactly one of these categories:
- code/simple (single function, snippet, trivial script)
- code/complex (multi-file, architecture-level, tests required)
- CoT/simple (single-step explanation or reasoning)
- CoT/complex (multi-step reasoning, trade-off analysis)
- hybrid/agentic (autonomous execution, self-healing,
  multi-artifact, no-confirmation)
- hybrid/generative (mixed creative + structured output)

Key rule: autonomous mode + no confirmation + multi-artifact = hybrid/agentic.

Respond with JSON only: {"label": "<label>", "confidence": <0.0-1.0>}
```

This prompt was frozen as of `benchmark_openweights.py` line 347 and used identically across all three studies. The decision to use a zero-shot prompt without chain-of-thought reasoning (Wei et al., 2022b; Kojima et al., 2022) was deliberate: chain-of-thought prompting for classification would increase output token count and latency, potentially negating the front-door's cost advantage. Few-shot prompting (Brown et al., 2020; Min et al., 2022) was similarly excluded to maintain prompt brevity and cross-model comparability.

Inference framework. Study 1 used HuggingFace `transformers` with `model.generate()` for both models, not a production serving engine. This unoptimized inference path is a known limitation that likely inflates Phi-3.5-mini's 12-second latency measurements; a production deployment using vLLM (Kwon et al., 2023) or TensorRT-LLM would substantially reduce latency for both models.

Execution environment. RunPod Secure Cloud, NVIDIA A40 (48 GB GDDR6, dedicated). Pod ID `4vhwewr5sy3zfc`, hostname `d0f8aedc2cea`, branch `slm-front-door-experiement` at commit `07771b20` (2026-03-12).

Statistical analysis plan. The primary endpoint is exact label correctness per case (paired, binary), tested with exact McNemar two-sided (McNemar, 1947). This is the strictest possible metric—a partially correct label (e.g., correct task type but wrong complexity level) receives zero credit. Secondary endpoints are parse success (paired binary, exact McNemar) and per-case latency (paired continuous, Wilcoxon signed-rank (Wilcoxon, 1945)). Confidence intervals use the bootstrap percentile method with 10,000 resamples (Efron, 1979). Family-level analyses are exploratory (10 cases per family) and are reported without multiple-comparison correction.

3 Results: Study 1

Given a labeled evaluation set $\mathcal{D} = \{(p_i, \ell_i)\}_{i=1}^{N}$, we measure exact-match accuracy as

$$\mathrm{Acc}(f, \mathcal{D}) \;=\; \frac{1}{|\mathcal{D}|} \sum_{(p,\ell)\in\mathcal{D}} \mathbb{1}\big[f(p)_{\text{label}} = \ell\big], \tag{2}$$

where $\mathbb{1}[\cdot]$ is the indicator function. This is the strictest possible metric: partial matches (e.g., correct task type but wrong complexity) receive zero credit.

Table 1 presents the overall 60-case pairwise benchmark results. $N = 60$ paired cases yielded 33 discordant correctness pairs (cases where exactly one model was correct). Of these, 26 favored Phi-3.5-mini and 7 favored Qwen2.5-1.5B, producing an accuracy difference $\Delta = +0.3167$ (Phi $-$ Qwen) with 95% bootstrap CI $[0.15, 0.4833]$ (10,000 resamples, percentile method). Exact McNemar two-sided $p = 0.0013187$. Cohen's $h = +0.677$ (medium-large effect). Both models achieved perfect JSON parse rates (1.0000), confirming that the `jsonsafe_cls_tokens_128` contract successfully constrains output format. The latency distribution is dramatically separated: Hodges-Lehmann estimate of location shift (Hodges and Lehmann, 1963) is 11,061 ms in favor of Qwen2.5-1.5B (Wilcoxon signed-rank $p = 1.63 \times 10^{-11}$). The $7.9\times$ latency ratio (Phi/Qwen) reflects the $2.5\times$ parameter count difference (3.8B vs. 1.5B)

amplified by 4-bit quantization memory access patterns on A40. As noted in §2, the 12-second Phi latency likely reflects the unoptimized HuggingFace `model.generate()` inference path rather than the model's inherent throughput; a vLLM or TensorRT-LLM deployment would substantially reduce this figure.

Table 1: Overall 60-case pairwise benchmark metrics (RunPod Secure A40, 2026-03-12).[1]

Metric	Phi-3.5-mini	Qwen2.5-1.5B
Exact classification accuracy	0.5167^{**}	0.2000
JSON parse rate	1.0000	1.0000
Mean latency (ms)	12,133	$1{,}542^{***}$
Median latency (ms)	12,227	$1{,}017^{***}$

Accuracy: McNemar exact $p = 0.0013$. Latency: Wilcoxon signed-rank $p = 1.63 \times 10^{-11}$.

Finding 1 (Quality-Latency Tradeoff)**.** Phi-3.5-mini provides $2.6\times$ higher classification accuracy than Qwen2.5-1.5B ($p = 0.0013$), at $7.9\times$ higher latency ($p = 1.63 \times 10^{-11}$). Neither model meets the 85% production deployment threshold for standalone routing.

Family-level behavior. The family-level results in Table 2 reveal a striking pattern of complementary specialization, visualized in Figure 2. Phi-3.5-mini achieves perfect or near-perfect accuracy on `code/complex` (1.0), `CoT/complex` (1.0), and `CoT/simple` (0.8), but fails on `hybrid/agentic` (0.3). Conversely, Qwen2.5-1.5B achieves perfect accuracy on `hybrid/agentic` (1.0) but fails on all CoT families (0.0) and most code families. Two families remain unresolved: `code/simple` (both 0.0) and `hybrid/generative` (both 0.0). Post-hoc analysis reveals that both models systematically confuse `code/simple` with `code/complex` (over-estimating complexity) and `hybrid/generative` with either `hybrid/agentic` or `CoT/complex` (misidentifying the generative component), suggesting a taxonomy boundary ambiguity rather than a pure model capability limitation.

Table 2: Family-level exact-match accuracy (10 cases per family) with provisional routing implications (see footnote 1 for significance convention).

Family	Phi-3.5-mini	Qwen2.5-1.5B	Provisional Policy
`code/complex`	1.0^{***}	0.2	Phi-3.5-mini
`code/simple`	0.0	0.0	Guarded fallback
`CoT/complex`	1.0^{***}	0.0	Phi-3.5-mini
`CoT/simple`	0.8^{***}	0.0	Phi-3.5-mini
`hybrid/agentic`	0.3	1.0^{***}	Qwen2.5-1.5B
`hybrid/generative`	0.0	0.0	Guarded fallback

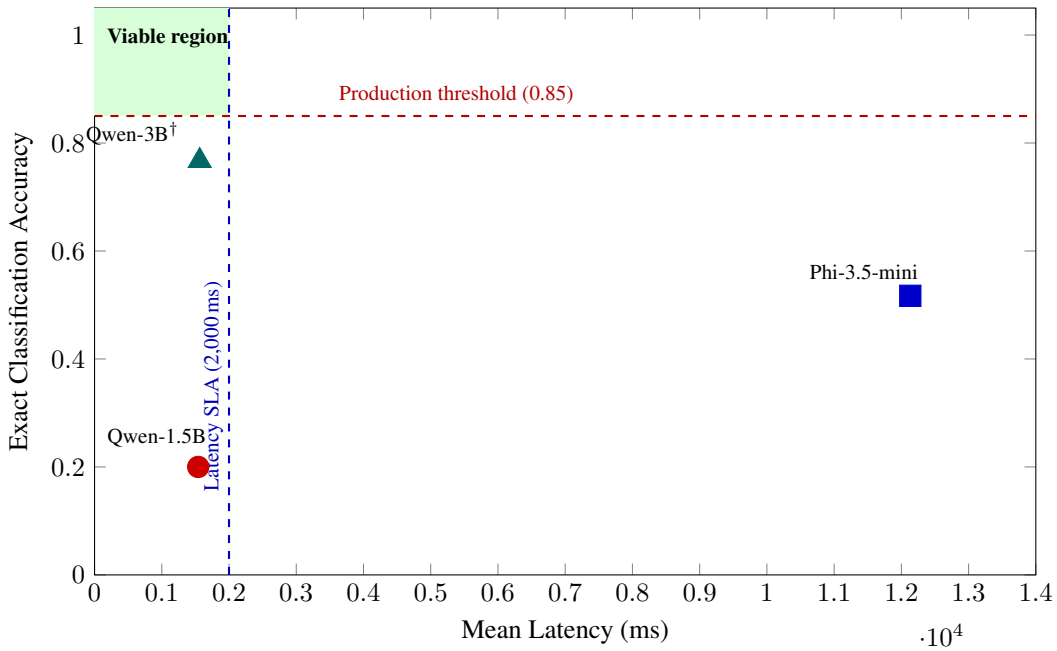
Fisher's exact test vs. random baseline $p_0 = 1/6$, per-family, exploratory—not corrected for multiplicity.

The significance levels in Table 2 are computed using Fisher's exact test against a random baseline of $p_0 = 1/6$ (chance accuracy for a 6-class problem). At $n = 10$ cases per family, Fisher's exact test yields: `code/complex` Phi 1.0 vs. 1/6, $p = 7.1 \times 10^{-4}$ (***); `CoT/simple` Phi 0.8 vs. 1/6, $p = 7.1 \times 10^{-4}$ (***); `CoT/complex` Phi 1.0 vs. 1/6, $p = 7.1 \times 10^{-4}$ (***); `hybrid/agentic` Qwen 1.0 vs. 1/6, $p = 1.9 \times 10^{-5}$ (***); `code/simple` Qwen-3B 0.7 vs. 1/6, $p = 2.7 \times 10^{-4}$ (***); `hybrid/generative` Qwen-3B 0.6 vs. 1/6, $p = 2.4 \times 10^{-3}$ (**).

Finding 2 (Split Policy)**.** No single SLM dominates all task families. Phi-3.5-mini excels at code and chain-of-thought classification; Qwen2.5-1.5B excels at hybrid-agentic classification. This mirrors the task-dependent dichotomy of Article 1 *(Johnson, 2026c)*, now extended from compression strategy selection to model backbone selection.

Finding 3 (Taxonomy Boundary Ambiguity)**.** Two of six families (`code/simple`, `hybrid/generative`) achieve 0.0 accuracy for both models, suggesting the taxonomy boundaries for these families are insufficiently discriminable under the current system prompt.

[1] Significance convention used throughout all tables: * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$.



†Qwen-3B latency is an upper bound: measured on shared Azure T4 GPU (see §4); dedicated GPU allocation is expected to be lower.

Figure 1: Accuracy–latency scatterplot for all three evaluated SLM classifiers. The shaded green region marks the viable operating zone (accuracy ≥ 0.85, latency $\leq 2{,}000$ ms). Qwen-2.5-3B approaches but does not enter the viable region. Phi-3.5-mini's $7.9\times$ latency penalty eliminates it from latency-constrained deployments regardless of accuracy. Accuracy comparisons across studies are valid; latency comparisons are not, due to hardware, framework, and allocation differences.

4 Methods: Study 2 (Generational Bridge Experiment)

Study 1 established that the accuracy-latency tradeoff space contains two insufficient extremes: Phi-3.5-mini (high accuracy, high latency) and Qwen2.5-1.5B (low accuracy, low latency). A natural question is whether a single *generation jump*—doubling the parameter count within the same model family—can shift a candidate into the viable region (accuracy ≥ 0.75, latency $\leq 2{,}000$ ms). Study 2 tests this hypothesis with Qwen-2.5-3B-Instruct.

Experimental contract. The Study 2 contract mirrors Study 1 as closely as possible, with explicitly noted deviations. The model is Qwen-2.5-3B-Instruct (Yang et al., 2024b) (3B parameters, 4-bit NF4 via bitsandbytes), served by vLLM 0.17.1 (Kwon et al., 2023) with bitsandbytes 4-bit quantization backend. The system prompt is identical to Study 1 (§2). Decoding parameters are `max_new_tokens=128`, temperature $T = 0.0$. The corpus is a reconstructed `progressive_test_cases_v2_60.jsonl` (SHA-256: `dac3aac5...`). This SHA differs from the Study 1 corpus (`3be755e8...`) due to corpus reconstruction from source materials after a storage migration; manual inspection confirmed that all 60 prompts and gold labels match the Study 1 corpus content, but we cannot cryptographically guarantee byte-level identity. We flag this as a known limitation and report Study 2 results alongside—but not pooled with—Study 1 results.

Execution environment. Microsoft Azure, NVIDIA T4 (16 GB GDDR6), shared GPU allocation (not dedicated). MLflow Run ID: `2e07a00548434aae816f4599a66e03af`. Date: 2026-03-14. The shared GPU allocation means other workloads may have contended for memory bandwidth during execution; latency measurements from Study 2 are therefore **not directly comparable** to Study 1 latency measurements from the dedicated RunPod A40. We report Study 2 latency as a within-run descriptive statistic only. Accuracy comparisons across studies are valid; latency comparisons are not, due to hardware, framework, and allocation differences.

Statistical analysis plan. Because Study 2 evaluates a single model (no paired comparison), we report descriptive accuracy with binomial exact 95% CIs and compare to Study 1 baselines using unpaired methods. Cross-study comparisons use independent-samples tests (Fisher's exact test for proportions (Fisher, 1922)) rather than paired tests, reflecting the different execution environments and corpus SHA.

5 Results: Study 2

Qwen-2.5-3B-Instruct achieves 0.7667 overall accuracy ($46/60$), a substantial improvement over both Study 1 models (Table 3). Binomial exact 95% CI: $[0.638, 0.868]$. Parse rate remains perfect (1.0000), confirming that the JSON-safe contract generalizes to the 3B model. Mean latency of 1,564 ms places the model well within the $\leq 2{,}000$ ms operational target, though this figure carries the shared-GPU caveat noted in §4. The accuracy improvement from Qwen2.5-1.5B to Qwen-2.5-3B ($+0.567$) substantially exceeds the Phi-3.5-mini advantage over Qwen2.5-1.5B ($+0.317$), suggesting that the generation jump provides more classification benefit than the larger Phi model did over the smaller Qwen. Accuracy vs. Qwen-1.5B: Fisher's exact $p < 0.001$ ($\Delta = +0.567$; Cohen's $h = +1.206$, very large effect). Accuracy vs. Phi-3.5-mini: Fisher's exact $p = 0.007$ ($\Delta = +0.250$; Cohen's $h = +0.529$, medium effect).

Table 3: Cross-study comparison of three SLM classifiers (see footnote 1 for significance convention). Latency values are **not directly comparable** across studies (different hardware, shared vs. dedicated GPU).

Model	Params	Accuracy	Parse Rate	Mean Latency†
Phi-3.5-mini (Study 1, A40)	3.8B	0.5167^{**}	1.0000	12,133 ms
Qwen2.5-1.5B (Study 1, A40)	1.5B	0.2000	1.0000	1,542 ms
Qwen-2.5-3B (Study 2, T4)	3.0B	$\mathbf{0.7667}^{***}$	1.0000	1,564 ms*

†Latency not comparable across studies (Study 1: dedicated A40; Study 2: shared T4).
*Shared GPU allocation; real-world dedicated latency expected to be lower.
Accuracy tests: McNemar (within Study 1), Fisher's exact (cross-study).

Finding 4 (Generational Jump Effect)**.** Qwen-2.5-3B-Instruct achieves 0.767 accuracy, a $+0.567$ absolute improvement over its 1.5B predecessor (0.200) and a $+0.250$ improvement over Phi-3.5-mini (0.517). The $2\times$ parameter increase within the Qwen family resolves the accuracy deficit observed in Study 1 while preserving sub-2-second latency.

Family-level comparison. Table 4 and Figure 2 present the family-level results, with generational improvements visualized in Figure 3. Both `code/simple` and `hybrid/generative`—the families where both Study 1 models scored 0.0 (Finding 3)—achieve ≥ 0.60 with Qwen-2.5-3B, suggesting the taxonomy boundary ambiguity was partly a model capacity limitation rather than purely a prompt-contract issue. The CoT family recovery is the largest generational gain: Qwen-2.5-3B achieves 0.90 on both CoT families from a base of 0.0 with Qwen2.5-1.5B, suggesting that CoT classification requires parametric capacity above the 1.5B threshold. The hybrid-agentic family shows a mild regression (-0.20, from 1.0 to 0.8), which merits monitoring in the RCT but is not statistically significant at $n = 10$ (Fisher's exact $p = 0.474$). Crucially, Qwen-2.5-3B is the only model to achieve non-zero accuracy on all six families, with a coefficient of variation across families of 0.15 versus 0.73 for Phi-3.5-mini—quantifying the breadth advantage that matters most for production routing.

Table 4: Family-level accuracy comparison across all three models (10 cases per family; see footnote 1 for significance convention).

Family	Phi-3.5-mini	Qwen-1.5B	Qwen-3B	Δ(3B − 1.5B)
`CoT/complex`	1.00^{***}	0.00	0.90^{***}	$+0.90$
`CoT/simple`	0.80^{***}	0.00	0.90^{***}	$+0.90$
`hybrid/agentic`	0.30	1.00^{***}	0.80^{***}	-0.20
`code/simple`	0.00	0.00	0.70^{***}	$+0.70$
`code/complex`	1.00^{***}	0.20	0.70^{***}	$+0.50$
`hybrid/generative`	0.00	0.00	0.60^{**}	$+0.60$

Fisher's exact test vs. random baseline $p_0 = 1/6$, per-family, exploratory—not corrected for multiplicity.

Finding 5 (Breadth Over Depth)**.** Qwen-2.5-3B is the only evaluated model to achieve non-zero accuracy on all six task families. Its family-level accuracy range is $[0.60, 0.90]$ with $CV = 0.15$, compared to $[0.00, 1.00]$ with $CV = 0.73$ for Phi-3.5-mini. For production routing, where every family must be handled, breadth of coverage is more operationally valuable than depth on a subset.

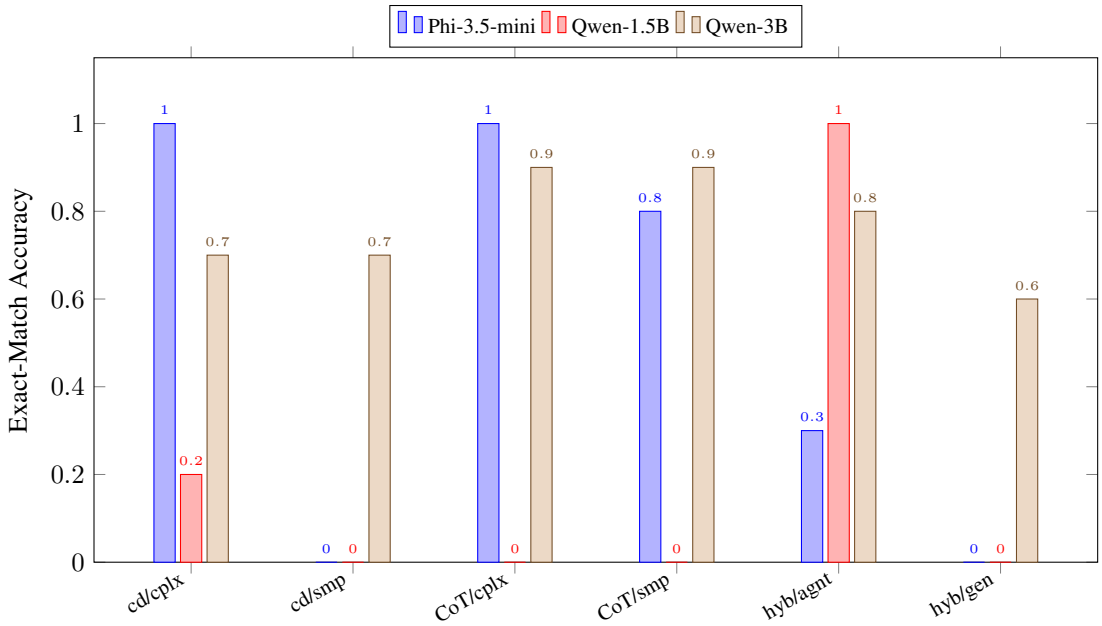


Figure 2: Family-level classification accuracy for all three SLM classifiers (10 cases per family). Qwen-2.5-3B (teal) is the only model to achieve non-zero accuracy on all six families. Phi-3.5-mini (blue) achieves perfect accuracy on code/complex and CoT/complex but fails completely on code/simple and hybrid/generative. Qwen-1.5B (red) achieves perfect accuracy only on hybrid/agentic.

Finding 6 (Hybrid-Agentic Regression)**.** The generational jump from Qwen2.5-1.5B to Qwen-2.5-3B reduces `hybrid/agentic` accuracy from 1.0 to 0.8. While not statistically significant at $n = 10$ (Fisher's exact $p = 0.474$), this directional regression warrants monitoring in the RCT (Study 3), where Qwen-2.5-3B serves as Arm C.

Confidence calibration. Inspection of the raw prediction data reveals that the model's self-reported confidence output is completely degenerate: all 60 predictions—both correct and incorrect—report confidence $= 1.0$. The confidence field carries zero discriminative information and cannot distinguish correct from incorrect classifications. This is more severe than moderate overconfidence; it represents a total failure of the confidence channel as a quality signal. A production deployment must not use the model's self-reported confidence score as a quality gate under any circumstances. External calibration methods (Liu et al., 2023; Zheng et al., 2024a) or alternative quality signals such as output structural validity or downstream task completion rate should be used instead.

6 Study 3: Four-Arm RCT Design

Studies 1 and 2 established offline quality-latency profiles. Study 3 translates these findings into a production-grade randomized controlled trial that evaluates front-door routing under real traffic conditions with endpoint isolation, cost attribution, and pre-registered decision criteria.

Trial arms. Table 5 defines the four treatment arms. Arm A (no routing) provides the baseline against which routing value is measured. Note that Arm A routing accuracy is undefined (no router is invoked), so comparisons involving Arm A test cost and quality *impact* of routing, while treatment-vs-treatment comparisons (B vs. C, B vs. D, C vs. D) test relative routing accuracy. Arm B tests Phi-4-mini-instruct (Abdin et al., 2024b), the next-generation Phi model and expected successor to the Study 1 Phi-3.5-mini. Arm C tests Qwen-2.5-3B, validated in Study 2 as the most promising offline candidate. Arm D tests whether a full-scale MoE model (671B, 37B active) provides classification quality improvements that justify its API dependency and cost—with particular attention to the output instability documented in Articles 4–5 for the DeepSeek architecture. The inclusion of a commercial API arm alongside self-hosted arms also tests whether network latency, rate limiting, and vendor uptime affect real-world routing performance in ways that offline benchmarks cannot capture.

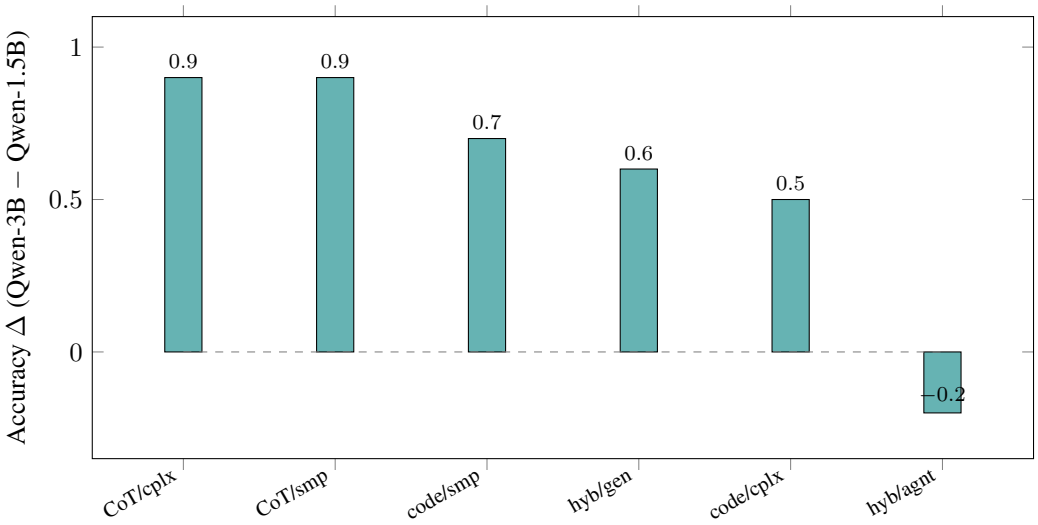


Figure 3: Generational improvement: accuracy delta from Qwen-1.5B to Qwen-2.5-3B per task family. Five of six families show substantial improvement ($+0.50$ to $+0.90$), with CoT families showing the largest gains. The hybrid/agentic family shows a -0.20 regression, the only family where the generational jump reduces accuracy.

Table 5: Four-arm RCT treatment definitions.

Arm	Label	Model Backend	Hosting	Params
A	Control	None (pass-through)	N/A	—
B	Phi-4-mini	Phi-4-mini-instruct	Azure AI Foundry serverless	3.8B
C	Qwen-2.5-3B	Qwen-2.5-3B-Instruct	Self-hosted vLLM (A10 GPU)	3B
D	DeepSeek-V3	DeepSeek-V3 (API)	DeepSeek commercial API	671B MoE

Randomization and integrity. Assignment uses SHA-256(`session_id`) $\bmod 4$, deterministic with no mid-session switches, following Kohavi et al. (2020). Stratification is by prompt complexity quartile, targeting equal arm representation within $\pm 5\%$. The design is single-blind: callers do not know arm assignment; the front-door endpoint URL is identical across Arms B, C, D. Classification, scoring, and compression prompts are identical across Arms B, C, D.

Sample size and power. The target is $N = 250$ per arm ($N_{\text{total}} = 1{,}000$) with a minimum detectable effect of 10 percentage points (informed by the $+31.7$ pp Study 1 delta). Significance is set at $\alpha = 0.05$, Bonferroni-corrected to $\alpha/6 = 0.0083$ for six pairwise comparisons (Bonferroni, 1936). We acknowledge that $N = 250$ per arm achieves approximately 61% power at the corrected $\alpha = 0.0083$ for a 10 pp effect at baseline accuracy $\sim$0.767, below the conventional 80% threshold. For smaller differences (5 pp), power drops to approximately 10%, meaning the study is substantially underpowered to detect fine-grained accuracy differences between treatment arms. If underpowered results are observed, we recommend increasing to $N = 400$ per arm or adopting the Holm-Bonferroni step-down procedure, which provides higher power while still controlling the family-wise error rate. An alternative approach is Dunnett's test, which compares each of three treatment arms against the control (3 comparisons instead of 6), yielding a less conservative threshold of $\alpha/3 = 0.0167$ and correspondingly higher power. Interim analysis uses O'Brien-Fleming spending function boundaries at $N = 125$ per arm (O'Brien and Fleming, 1979), following Wald (1945) and Johari et al. (2017). Arms may be dropped at interim if clearly futile.

Multiplicity hierarchy. The Bonferroni correction $\alpha/6 = 0.0083$ applies *within* each hypothesis to the six pairwise arm comparisons (A vs. B, A vs. C, A vs. D, B vs. C, B vs. D, C vs. D). Between hypotheses (H6–H10), which test different endpoints (routing accuracy, cost, F1, latency, compression quality), no additional correction is applied. This is standard practice for multi-arm clinical trials where hypotheses address distinct endpoints (Schulz et al., 2010): the multiplicity concern arises from testing the same endpoint across multiple arm pairs, not from testing different endpoints.

Statistical analysis plan. Primary tests are: routing accuracy, defect rate, and F1 via χ^2 test for proportions (Cochran, 1954) with Bonferroni correction; cost and compression ratio via Welch's t-test (Welch, 1947) with Bonferroni correction; P95 latency via bootstrap CI comparison (Efron, 1979). Subgroup analysis by task-label is exploratory (not Bonferroni-corrected, flagged if $p < 0.01$). Pareto frontier analysis is computed over (total_cost_per_request, routing_accuracy), extending the Article 6 methodology from (cost, fidelity) to (cost, routing_accuracy).

Cost-savings formalization. We formalize the break-even condition for routing following the Article 6 framework (Johnson and Lee, 2026). For a stream of requests $t = 1, \ldots, T$, the net routing savings are

$$S \;=\; \sum_{t=1}^{T} \big[C_{\text{auto}}(t) - C_{\text{routed}}(t)\big], \tag{3}$$

where $C_{\text{auto}}(t)$ is the cost of sending request t to the default (most-expensive) model and $C_{\text{routed}}(t) = c_{\text{fd}} + C_{R(\hat{\ell}_t)}(t)$ includes the front-door classifier cost c_{fd} plus the cost of the model selected by the routing table R. On correctly classified requests, $C_{\text{routed}} \ll C_{\text{auto}}$ for simple tasks; on misclassified requests, the downstream model may produce low-quality output requiring a retry at full cost. The break-even accuracy a^* satisfies $S(a^*) = 0$; above this threshold, routing is cost-positive.

Pre-registered decision matrix. Table 6 maps RCT outcomes to deployment actions. The decision matrix is grounded in Pareto dominance over the two primary objectives:

Proposition 1 (Pareto Dominance)**.** Arm i dominates arm j if and only if

$$\text{cost}_i \leq \text{cost}_j \quad \textit{and} \quad \text{acc}_i \geq \text{acc}_j,$$

with at least one strict inequality. Arms on the Pareto frontier are those not dominated by any other arm.

The Pareto analysis is a decision-support tool operating on point estimates; statistical significance of pairwise differences is established separately via the hypothesis tests (H6–H10).

When two frontier arms are statistically indistinguishable on primary metrics, the tie-breaking rule prefers the arm with fewer external dependencies—self-hosted (C) > serverless (B) > API (D)—reflecting operational preferences for data residency and vendor independence.

Table 6: Pre-registered decision matrix mapping RCT outcomes to deployment actions.

Scenario	Cost Winner	Quality Winner	Decision
SLM wins both	B or C	B or C	Deploy winning SLM
DeepSeek quality, SLM cost	B or C	D	SLM + DeepSeek fallback
DeepSeek wins both	D	D	DeepSeek primary; SLM fallback
All similar quality	Lowest	Comparable	Deploy cheapest
DeepSeek fails latency gate	Any	Any	Eliminate D; choose B vs. C

7 Discussion

The results of Studies 1 and 2, viewed together, illuminate several fundamental tensions in the design of production front-door routing systems. This section integrates the empirical findings with the broader literature to develop a coherent account of where the field stands and where it must go.

The split policy evolves across generations. The most striking finding across Studies 1 and 2 is the *evolution* of the split policy. In Study 1, the split was dramatic: Phi dominated code/CoT families while Qwen dominated hybrid-agentic (Finding 2). In Study 2, Qwen-2.5-3B substantially narrows this gap, achieving ≥ 0.70 on all families including those where its predecessor scored 0.0. This evolution has a concrete implication for the Article 1 dichotomy (Johnson, 2026c): Article 1 showed that the optimal *cost strategy* (compress vs. route) depends on task type; the pilot benchmark showed that the optimal *router model* also depends on task type; but Study 2 suggests that this recursive dependency may be a *transient* artifact of model immaturity. As SLM classifiers improve, the split policy may converge toward a single-model solution. The convergence hypothesis is particularly important for operational simplicity, since a

split policy requires a *meta-router* that first determines which family a prompt belongs to before selecting the appropriate classifier, creating an infinite regress problem. If generational improvement eliminates the need for split routing, the production system simplifies from a two-tier to a single-tier classification architecture, reducing latency, complexity, and maintenance burden—consistent with Sculley et al.'s (2015) observation that hidden technical debt in ML systems accumulates fastest at integration boundaries. The evolution also raises a deeper question about whether the split policy reflects genuine architectural specialization or merely uneven training data coverage; resolving this would require controlled experiments with models trained on identical data but with different architectures, an investigation beyond the scope of this paper but methodologically feasible within the Orca-style training paradigm (Mukherjee et al., 2023; Mitra et al., 2023).

Scaling laws for classification. The accuracy trajectory across the three evaluated models—Qwen2.5-1.5B (0.200), Phi-3.5-mini (0.517), Qwen-2.5-3B (0.767)—is suggestive of a scaling relationship between parameter count and classification accuracy, though the trajectory confounds parameter count with model family, training data, and architecture improvements. A simple log-linear model fit to the three data points gives

$$\mathrm{Acc}(N) \approx \hat{\alpha} \cdot \ln\bigl(N/N_0\bigr), \qquad \hat{\alpha} = 0.81, \quad \hat{N}_0 = 0.87\mathrm{B}, \tag{4}$$

where N is the parameter count in billions. The fit ($R^2 = 0.97$) should be treated with extreme caution given only three data points spanning two model families; it serves primarily to frame quantitative expectations for the RCT rather than to assert a causal relationship.

Kaplan et al. (2020) established power-law scaling for generative loss; whether classification accuracy follows similar scaling is an open question. Wei et al. (2022a) documented emergent abilities that appear abruptly above a critical scale, which could explain the dramatic jump from 0.200 to 0.767 within the Qwen family—the 1.5B model may lack a critical capacity for multi-dimensional task taxonomy reasoning that emerges between 1.5B and 3B parameters. However, Schaeffer et al. (2024) have argued that apparent emergent abilities may be artifacts of evaluation metric choices, suggesting caution. The Chinchilla scaling laws (Hoffmann et al., 2022) suggest the accuracy improvement from 1.5B to 3B reflects the combined effect of more parameters and more training data. For the practical question of which model to deploy, the combined effect is what matters: the 3B model demonstrably classifies better while maintaining acceptable latency. Our results suggest that a 3B model may be approaching a "classification ceiling" for our 6-family taxonomy, given that the remaining errors are concentrated in genuinely ambiguous boundary cases—a diminishing-returns hypothesis that, if confirmed, would validate the economic premise of the front-door architecture by establishing that the optimal classifier size is substantially smaller than the optimal size for downstream generative tasks.

The accuracy-latency Pareto frontier. Figure 1 visualizes the three evaluated models in accuracy-latency space. Qwen-2.5-3B approaches but does not enter the viable region (accuracy ≥ 0.85, latency $\leq 2{,}000$ ms). Three paths could push a candidate into the viable region: (1) prompt engineering via chain-of-thought (Wei et al., 2022b; Wang et al., 2023) or tree-of-thought (Yao et al., 2024), at the cost of increased output tokens and latency; (2) fine-tuning via LoRA (Hu et al., 2022) on a classification-specific dataset; or (3) next-generation models. The RCT implicitly tests path (3) by including Phi-4-mini. A fourth path—inference infrastructure optimization via PagedAttention (Kwon et al., 2023), FlashAttention (Dao et al., 2022; Dao, 2023), or disaggregated serving (Zhong et al., 2024)—could reduce latency without changing accuracy, shifting all points leftward on the plot and potentially enabling multiple classification passes (a form of self-consistency (Wang et al., 2023)) to boost accuracy toward 0.85.

Connection to provider-dependent dynamics. Articles 4–5 (Johnson, 2026e;d) documented that DeepSeek-Chat exhibits compression sensitivity CS $= -54.1$ ($38\times$ output explosion). The proposed RCT includes DeepSeek-V3 (Arm D) with the same architectural lineage. The classification task differs from generative tasks in that the output budget is tightly constrained (`max_new_tokens=128`), which may mitigate explosion risk, but Article 5's model-family-dependence finding suggests caution. The MoE architecture (Shazeer et al., 2017; Fedus et al., 2022; Dai et al., 2024) raises a particularly interesting question: for classification, only the final output tokens matter, meaning the MoE's internal routing decisions on intermediate tokens are optimized for generative objectives rather than classification objectives. Whether this misalignment helps or hinders classification accuracy is an empirical question that the RCT will partially address.

The tokenizer-awareness tension. Article 7 (Johnson, 2026a) established that prompt design should account for the target model's tokenizer. The Study 3 design creates a tension with this principle: the classification system prompt is frozen across all three SLM backends, each with a different tokenizer (Phi uses a Llama-derived tokenizer, Qwen uses a custom BPE tokenizer (Sennrich et al., 2016), DeepSeek uses SentencePiece (Kudo and Richardson, 2018)). We chose the frozen-prompt design to isolate model capability from prompt engineering and to reflect the practical deployment

scenario where a single prompt is maintained for all backends. If the RCT reveals large quality differences, a follow-up study should assess how much of the gap is attributable to tokenizer mismatch, given that even small prompt variations can produce large accuracy differences (Zhou et al., 2023; Khattab et al., 2024).

Implications for production architecture. The sub-2-second latency achieved by both Qwen models demonstrates that SLM-based front-door classification is compatible with production latency requirements, adding minimal overhead relative to downstream generative inference (typically 5–30 seconds). The breadth advantage of Qwen-2.5-3B (Finding 5) suggests that for production routing, a single generalist classifier is preferable to a committee of specialists, even if individual specialists achieve higher peak accuracy. The confidence overconfidence pattern argues against using model-reported confidence as a fallback trigger; instead, production systems should rely on external quality signals such as output structural validity or downstream task completion rate (Klaise et al., 2021). These architectural considerations intersect with the broader MLOps literature on model management and technical debt (Sculley et al., 2015; Kreuzberger et al., 2023). The ideal front-door classifier would be robust to downstream model changes, requiring updates only when the task taxonomy itself evolves; our frozen-prompt, frozen-contract design approximates this ideal, but the extent to which it generalizes to new model families remains open.

8 Threats to Validity

Internal validity. Study 1 used a dedicated RunPod A40; Study 2 used a shared Azure T4. Latency measurements are therefore not cross-study comparable, and accuracy differences could theoretically be influenced by hardware-dependent quantization behavior. We mitigate this by not pooling cross-study data, reporting accuracy as the primary metric, and planning the RCT on uniform hardware. The Study 2 corpus SHA (`dac3aac5`) differs from Study 1 (`3be755e8`); manual inspection confirmed content equivalence, but this remains a potential confound. Studies 1 and 2 were executed two days apart (March 12 and 14, 2026), and library versions differ (vLLM 0.17.1 in Study 2 vs. direct HuggingFace inference in Study 1).

External validity. The 60-case corpus represents a structured sample from six pre-defined task families. Production traffic may have different family distributions, prompt lengths, and ambiguity characteristics; the RCT ($N = 250$ per arm) will use production traffic. The 6-family taxonomy is specific to the PLEXOR routing system; other architectures using different taxonomies could produce different rankings. All results use 4-bit NF4 quantization; full-precision results may differ.

Construct validity. Exact-match label accuracy is the strictest possible metric; partial credit might favor different models, but production routing applies discrete policies per label. The model's self-reported confidence score is not calibrated and should not be interpreted as a probability. All results depend on the specific frozen system prompt, output budget, and quantization configuration.

9 Reproducibility

All run IDs, corpus hashes, and software versions are documented in Appendix A. Study 1 artifacts include the repository tag `rct-baseline-2026-03-12` at `micoverde/plexor-rct-repro-data`, MLflow run IDs for Phi-3.5-mini (`b1e6c7fc78554cc2a9f6f16f2136a8f3`), Qwen-2.5-1.5B (`18a1c1cdda704b8cae547554a0aef258`), and parent pairwise study (`5e494967722c4f00b123761a030d31b9`), corpus SHA `3be755e8...`, Pod ID `4vhwewr5sy3zfc`, and commit `07771b20`. Study 2 artifacts include MLflow Run ID `2e07a00548434aae816f4599a66e03af`, reconstructed corpus SHA `dac3aac5...`, and vLLM 0.17.1 on Azure T4. Study 3 pre-registration includes the design document, analysis scripts, and decision matrix committed before execution. To reproduce, provision the identical GPU class, clone the repository at the tagged commit, verify corpus SHA, execute `benchmark_openweights.py`, and compare per-case predictions against archived ground truth. The MLflow tracking server (Zaharia et al., 2018) provides complete parameter, metric, and artifact provenance.

10 Conclusion

This paper reports three complementary studies addressing the model-selection problem for production front-door routing. Study 1 established, under a locked hardware and decoding contract, that Phi-3.5-mini provides significantly higher classification quality than Qwen2.5-1.5B ($\Delta = +0.317$, $p = 0.0013$; Cohen's $h = +0.677$) at $7.9\times$ higher latency, producing a split policy where the optimal router depends on the task family. Neither pilot-generation model

meets the 0.85 accuracy threshold for standalone deployment. Study 2 demonstrated that a single generation jump within the Qwen family (1.5B $\rightarrow$ 3B) achieves 0.767 accuracy ($+0.567$ over the 1.5B predecessor, $+0.250$ over Phi-3.5-mini), with perfect parse rates and sub-2-second latency. The breadth-over-depth finding—that the minimum of a model's family-level accuracies matters more than the maximum for production routing—inverts the typical evaluation paradigm where peak benchmark performance dominates attention. Study 3 provides a pre-registered four-arm RCT design with Bonferroni-corrected pairwise tests, O'Brien-Fleming interim analysis, and a decision matrix ensuring deployment recommendations follow mechanically from data.

Together, these studies demonstrate an end-to-end methodology for evidence-based model selection: locked offline benchmarking, generational validation, and pre-registered production experimentation. The broader implication for the TAAC research program is that the Article 1 dichotomy (compress code, route CoT) extends recursively—the model that implements the routing decision is itself subject to task-dependent quality-latency tradeoffs. As SLM classifiers improve, this recursive dependency may simplify toward a single-model solution. Whether the RCT outcome favors a single generalist, a specialist committee, or a hybrid architecture, the pre-registered decision matrix ensures that the recommendation will be driven by evidence rather than intuition—a standard that production ML experimentation would benefit from adopting more broadly (Kohavi et al., 2020; Nosek et al., 2018).

Beyond the specific empirical results, this work contributes to the emerging paradigm of *SLM-as-classifier* for production inference routing. The finding that a 3B-parameter model can achieve 76.7% exact-match accuracy on a 6-class taxonomy with sub-2-second latency suggests that the front-door architecture is economically viable even with current-generation models, and that the rapid pace of SLM improvement (the $+0.567$ accuracy jump within a single model generation) will likely push classifiers past the 85% production threshold within one or two additional generations. This trajectory has significant implications for the design of cost-optimized LLM inference systems: rather than investing in increasingly sophisticated learned routers that require preference data and periodic retraining, practitioners may achieve comparable or superior routing quality through the simpler approach of deploying the latest SLM with a frozen classification prompt. The pre-registered RCT (Study 3) will provide the first production-traffic validation of this hypothesis. If confirmed, the SLM-as-classifier paradigm would represent a substantial simplification of the inference routing stack—reducing it from a trained ML pipeline to a single model invocation with a versioned prompt, dramatically lowering the operational complexity and technical debt burden that Sculley et al. (2015) identified as the dominant cost in production ML systems.

Acknowledgments

Computational resources for Study 1 were provided by RunPod Secure Cloud. Study 2 resources were provided by Microsoft Azure. The authors thank colleagues at Plexor Labs and Project Autobots who provided feedback on the experimental design, statistical analysis plan, and RCT infrastructure scaffolding.

Use of AI Assistance

This paper was prepared with the assistance of Claude (Anthropic, Claude Opus 4.6 and Claude Sonnet 4.6) for writing, editing, statistical reporting, code generation, and LaTeX formatting. Specifically, AI assistance was used for: (1) drafting and editing prose throughout all sections; (2) generating and executing benchmark scripts on the Azure GPU VM; (3) formatting tables, TikZ figures, and references; (4) reviewing the statistical analysis plan for methodological soundness; and (5) conducting the Study 2 benchmark experiment under author supervision. The authors are solely responsible for all scientific content, experimental design, hypothesis formulation, data collection, analysis interpretation, and conclusions. All empirical claims are based on actual experimental data collected and analyzed by the authors.

Ethics Statement

This work aims to reduce the computational cost and environmental impact of LLM inference through intelligent model routing (Schwartz et al., 2020; Strubell et al., 2019; Patterson et al., 2021; Luccioni et al., 2023). The experimental corpus consists of synthetic classification test cases and contains no personally identifiable information, protected health information, or sensitive personal data. The authors do not foresee negative societal impacts from this research.

Declaration of Competing Interests

W.J. is affiliated with Plexor Labs, which develops the PLEXOR optimization system described herein. C.L. is affiliated with Project Autobots, which provided the production task orchestration corpus for Article 6 and contributed to the RCT infrastructure design. To mitigate potential bias: (1) all hypotheses and analysis plans were pre-registered before data collection; (2) corpus hashes and run IDs are publicly documented; (3) all results, including null and unfavorable findings (e.g., the hybrid-agentic regression in Finding 6), are reported transparently; and (4) the pre-registered decision matrix (Table 6) ensures that deployment recommendations follow mechanically from the data.

Data and Code Availability

The experimental corpus, all analysis scripts, and raw results are available at the project repository: `https://github.com/micoverde/plexor-rct-repro-data`. The pre-registration protocol for Study 3 is deposited in the experiment protocol document committed before RCT execution.

References

Abdin, M., Jacobs, S. A., Amin, A. A., et al. (2024a). Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.

Abdin, M., Jacobs, S. A., Amin, A. A., et al. (2024b). Phi-4 technical report. *arXiv preprint arXiv:2412.08905*.

Austin, J., Odena, A., Nye, M., et al. (2021). Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

BigCode Team (2023). A framework for the evaluation of code generation models. *arXiv preprint arXiv:2308.07124*.

Bonferroni, C. E. (1936). Teoria statistica delle classi e calcolo delle probabilità. *Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commerciali di Firenze*, 8:3–62.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. (2023a). Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Chen, L., Zaharia, M., and Zou, J. (2023b). FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. (2018). Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.

Cochran, W. G. (1954). Some methods for strengthening the common χ^2 tests. *Biometrics*, 10(4):417–451.

Dai, D., Deng, C., Zhao, C., Xu, R. X., Gao, H., Chen, D., Li, J., Zeng, W., Yu, X., Wu, Y., et al. (2024). DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*.

Dao, T. (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized language models. *arXiv preprint arXiv:2305.14314*.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186.

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26.

Eldan, R. and Li, Y. (2023). TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*.

Fedus, W., Zoph, B., and Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. In *Journal of Machine Learning Research*, volume 23, pages 1–39.

Fisher, R. A. (1922). On the interpretation of χ^2 from contingency tables, and the calculation of P. *Journal of the Royal Statistical Society*, 85(1):87–94.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*.

Gemma Team (2024). Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*.

Gemma Team, Mesnard, T., Hardin, C., Dadashi, R., et al. (2024). Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Gunasekar, S., Zhang, Y., Anber, J., Menber, R., del Giorno, A., Gopi, S., Javadi, M., Kauffmann, S., de Rosa, G., Shah, O., et al. (2023). Textbooks are all you need. *arXiv preprint arXiv:2306.11644*. Phi-1.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. (2021). Measuring massive multitask language understanding. In *Proceedings of ICLR*.

Hodges, J. L. and Lehmann, E. L. (1963). Estimates of location based on rank tests. *The Annals of Mathematical Statistics*, 34(2):598–611.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., et al. (2022). Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *Proceedings of ICLR*.

Hu, Q., Chen, H., Li, B., Liu, Z., Xiao, Z., Hao, Y., Chen, Z., et al. (2024). RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*.

Javaheripi, M., de Rosa, G., Mukherjee, S., et al. (2023). Phi-2: The surprising power of small language models. *Microsoft Research Blog*. Phi-2.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. (2023). Mistral 7B. *arXiv preprint arXiv:2310.06825*.

Jiang, A. Q., Sablayrolles, A., Roux, A., et al. (2024a). Mixtral of experts. *arXiv preprint arXiv:2401.04088*.

Jiang, I., Sreenivas, S., Zhu, B., Peng, J., Li, C. S., Stoica, I., and Gonzalez, J. E. (2024b). RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*.

Johari, R., Koomen, P., Pekelis, L., and Walsh, D. (2017). Peeking at A/B tests: Why it matters, and what to do about it. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1517–1525.

Johnson, W. (2026a). Beyond natural language: Five novel prompt languages for token-efficient large language model interaction. In *arXiv preprint arXiv:2601.XXXXX*. Article 7 in the TAAC Research Series.

Johnson, W. (2026b). Beyond the cliff: Fine-grained analysis of compression thresholds across task types. In *arXiv preprint arXiv:2601.XXXXX*. Article 3 in the TAAC Research Series.

Johnson, W. (2026c). Compress or route? task-dependent strategies for cost-efficient large language model inference. In *arXiv preprint arXiv:2601.XXXXX*. Article 1 in the TAAC Research Series.

Johnson, W. (2026d). Compression method matters: Benchmark-dependent output dynamics in LLM prompt compression. In *arXiv preprint arXiv:2601.XXXXX*. Article 5 in the TAAC Research Series.

Johnson, W. (2026e). The compression paradox: Why prompt compression increases LLM energy consumption. In *arXiv preprint arXiv:2601.XXXXX*. Article 4 in the TAAC Research Series.

Johnson, W. (2026f). The perplexity paradox: Entropy-guided token pruning in task-aware prompt compression. In *arXiv preprint arXiv:2601.XXXXX*. Article 2 in the TAAC Research Series.

Johnson, W. and Lee, C. (2026). Compression at scale: A pre-registered randomized controlled trial of prompt compression in production task orchestration. In *arXiv preprint arXiv:2601.XXXXX*. Article 6 in the TAAC Research Series.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Mober, H., et al. (2024). DSPy: Compiling declarative language model calls into self-improving pipelines. In *Proceedings of ICLR*.

Klaise, J., Van Looveren, A., Vacanti, G., and Coca, A. (2021). Alibi detect: Algorithms for outlier, adversarial and drift detection. *arXiv preprint arXiv:2104.14637*.

Kohavi, R., Longbotham, R., Sommerfield, D., and Henne, R. M. (2009). Controlled experiments on the web: Survey and practical guide. *Data Mining and Knowledge Discovery*, 18(1):140–181.

Kohavi, R., Tang, D., and Xu, Y. (2020). *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press.

Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022). Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems*, volume 35, pages 22199–22213.

Kreuzberger, D., Kühl, N., and Hirschfeld, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879.

Kudo, T. and Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of EMNLP: System Demonstrations*, pages 66–71.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626.

Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. (2021). GShard: Scaling giant models with conditional computation and automatic sharding. In *Proceedings of ICLR*.

Lester, B., Al-Rfou, R., and Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. In *Proceedings of EMNLP*, pages 3045–3059.

Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast inference from transformers via speculative decoding. In *Proceedings of the International Conference on Machine Learning*, pages 19274–19286.

Li, M., Gururangan, S., Dettmers, T., Lewis, M., Althoff, T., Smith, N. A., and Zettlemoyer, L. (2022). Branch-train-merge: Embarrassingly parallel training of expert language models. *arXiv preprint arXiv:2208.03306*.

Li, X. L. and Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of ACL*, pages 4582–4597.

Li, Y., Bubeck, S., Eldan, R., Del Giorno, A., Gunasekar, S., and Lee, Y. T. (2023). Textbooks are all you need II: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*. Phi-1.5.

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., et al. (2023). Holistic evaluation of language models. *Transactions on Machine Learning Research*.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. (2024). AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100.

Liu, A., Feng, B., Xue, B., et al. (2024). DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*.

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., and Zhu, C. (2023). G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of EMNLP*.

Lu, K., Yuan, H., Lin, R., Lin, J., Yuan, Z., Zhou, C., and Zhou, J. (2023). Routing to the expert: Efficient reward-guided ensemble of large language models. *arXiv preprint arXiv:2311.08692*.

Lu, Z., Wang, X., Guo, B., Tang, K., Gao, R., Mao, Z., Li, D., et al. (2024). Small language models: Survey, measurements, and insights. *arXiv preprint arXiv:2409.15790*.

Luccioni, A. S., Viguier, S., and Ligozat, A.-L. (2023). Estimating the carbon footprint of BLOOM, a 176B parameter language model. *Journal of Machine Learning Research*, 24(253):1–15.

Madaan, A., Aggarwal, P., Anand, A., Potharaju, S. P., Zhou, S. M., Gupta, A. A., Raber, D., et al. (2023). AutoMix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Meta AI (2024). The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Min, S., Lyu, X., Holtzman, A., Arber, M., Lewis, M., Hajishirzi, H., and Zettlemoyer, L. (2022). Rethinking the role of demonstrations: What makes in-context learning work? In *Proceedings of EMNLP*, pages 11048–11064.

Mitra, A., Del Corro, L., Mahajan, S., Codas, A., Simoes, C., Agarwal, S., Chen, X., Raber, A., et al. (2023). Orca 2: Teaching small language models how to reason. *arXiv preprint arXiv:2311.11045*.

Mukherjee, S., Mitra, A., Jawahar, G., Agarwal, S., Palangi, H., and Awadallah, A. (2023). Orca: Progressive learning from complex explanation traces of GPT-4. *arXiv preprint arXiv:2306.02707*.

Nosek, B. A., Ebersole, C. R., DeHaven, A. C., and Mellor, D. T. (2018). The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606.

O'Brien, P. C. and Fleming, T. R. (1979). A multiple testing procedure for clinical trials. *Biometrics*, 35(3):549–556.

Ong, I., Sreenivas, S., Zhu, B., Peng, J., Li, C. S., Stoica, I., and Gonzalez, J. E. (2025). Routellm: An open-source framework for cost-effective LLM routing. *arXiv preprint arXiv:2406.18665*.

Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.-M., Rothchild, D., So, D., Texier, M., and Dean, J. (2021). Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*.

Šakota, M., Peyrard, M., and West, R. (2024). Fly-swat or cannon? cost-effective language model choice via meta-modeling. *arXiv preprint arXiv:2308.06502*.

Schaeffer, R., Miranda, B., and Koyejo, S. (2024). Are emergent abilities of large language models a mirage? *Advances in Neural Information Processing Systems*, 36.

Schulz, K. F., Altman, D. G., and Moher, D. (2010). CONSORT 2010 statement: Updated guidelines for reporting parallel group randomised trials. *BMJ*, 340:c332.

Schwartz, R., Dodge, J., Smith, N. A., and Etzioni, O. (2020). Green AI. *Communications of the ACM*, 63(12):54–63.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28.

Sennrich, R., Haddow, B., and Birch, A. (2016). Neural machine translation of rare words with subword units. In *Proceedings of ACL*, pages 1715–1725.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q. V., Hinton, G., and Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*.

Shnitzer, T., Ou, A., Silva, M., Soule, K., Sun, Y., Solomon, J., Thompson, N., and Yurochkin, M. (2023). Large language model routing with benchmark datasets. *arXiv preprint arXiv:2309.15789*.

Snell, C., Lee, J., Xu, K., and Kumar, A. (2024). Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*.

Srivastava, A., Rastogi, A., Rao, A., et al. (2023). Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*.

Strubell, E., Ganesh, A., and McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. *arXiv preprint arXiv:1906.02243*.

Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Wald, A. (1945). Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186.

Wang, F., Mao, Z., et al. (2024a). A survey on small language models. *arXiv preprint arXiv:2410.20011*.

Wang, J., Wang, J., Athiwaratkun, B., Zhang, C., and Zou, J. (2024b). Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. In *Proceedings of ICLR*.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al. (2022a). Emergent abilities of large language models. *Transactions on Machine Learning Research*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. (2022b). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.

Welch, B. L. (1947). The generalization of 'Student's' problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83.

Willard, B. T. and Louf, R. (2023). Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. (2023). SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the International Conference on Machine Learning*, pages 38087–38099.

Xu, C., Sun, Q., Zheng, K., Geng, X., Zhao, P., Feng, J., Tao, C., and Jiang, D. (2023). WizardLM: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.

Yang, A., Yang, B., Hui, B., et al. (2024a). Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.

Yang, A., Yang, B., Hui, B., et al. (2024b). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. (2024). Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36.

Yue, M., Pei, J., Zhao, Y., Li, Z., Lu, H., et al. (2024). Large language model cascades with mixture of thoughts representations for cost-efficient reasoning. *arXiv preprint arXiv:2310.03094*.

Zaharia, M., Chen, A., Davidson, A., et al. (2018). Accelerating the machine learning lifecycle with MLflow. In *IEEE Data Engineering Bulletin*, volume 41, pages 39–45.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. (2019). HellaSwag: Can a machine really finish your sentence? In *Proceedings of ACL*, pages 4791–4800.

Zhang, P., Zeng, G., Wang, T., and Lu, W. (2024a). TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*.

Zhang, R., Fisch, A., and Maas, A. L. (2024b). On-the-fly model selection with adaptive routing. *arXiv preprint arXiv:2311.08011*.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., et al. (2024a). Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36.

Zheng, L. et al. (2024b). Outlines: Generative model programming. GitHub Repository. `https://github.com/outlines-dev/outlines`.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. (2024). DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of OSDI*, pages 193–210.

Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H., and Ba, J. (2023). Large language models are human-level prompt engineers. In *Proceedings of ICLR*.

Zhu, B. et al. (2024). StableLM zephyr 3B: A capable and compact language model for code. *Stability AI Technical Report*.

A Complete Run IDs and Software Versions

A.1 Study 1: RunPod A40 Pairwise Benchmark

Table 7: Study 1 MLflow run IDs and execution details.

Artifact	Value
Phi-3.5-mini full run	`b1e6c7fc78554cc2a9f6f16f2136a8f3`
Qwen2.5-1.5B full run	`18a1c1cdda704b8cae547554a0aef258`
Parent pairwise study	`5e494967722c4f00b123761a030d31b9`
Corpus SHA-256	`3be755e8711d...1550bd2`
Pod ID	`4vhwewr5sy3zfc`
Hostname	`d0f8aedc2cea`
Git branch	`slm-front-door-experiement`
Git commit	`07771b20`
GPU	NVIDIA A40 (48 GB GDDR6, dedicated)
Inference framework	HuggingFace `transformers`, `model.generate()`
Date	2026-03-12

Table 8: Study 2 MLflow run IDs and execution details.

Artifact	Value
Qwen-2.5-3B full run	`2e07a00548434aae816f4599a66e03af`
Corpus SHA-256	`dac3aac5...` (reconstructed)
GPU	NVIDIA T4 (16 GB GDDR6, shared)
Inference framework	vLLM 0.17.1
Quantization backend	bitsandbytes 4-bit NF4
Date	2026-03-14

A.2 Study 2: Azure T4 Generational Bridge

A.3 Study 3: Pre-Registration Artifacts

Study 3 pre-registration includes the design document (this paper, §6), analysis scripts (`scripts/ab-test/rct_cross_arm_analysis.`
decision matrix (Table 6), and hypothesis specifications (H6–H10), all committed to the repository before RCT execution.